

LinkaBoo — Product Specification

Version: 1.0 (MVP) **Last Updated:** April 2026 **Status:** Active Development

The Problem

File sharing is broken in three specific ways:

1. **Cost** — Every major solution (WeTransfer, Dropbox, Google Drive) eventually charges for storage or transfer volume. Users pay for infrastructure they shouldn't need.
2. **Latency** — Uploading a large file to a server only so someone else can download it from that same server is redundant data transfer. If two machines can talk to each other directly, they should.
3. **UX Friction** — The current workflow is: find a service → sign in → upload → wait → copy link → send link → recipient downloads. That's 6+ steps for what should be 2.

LinkaBoo solves all three. Files move directly between machines using the Magic Wormhole protocol. No storage. No upload wait. The UX is drag, enter an email or name, done.

What LinkaBoo Is

A native macOS menubar application that enables peer-to-peer file and folder transfer with a branded shareable link. Files are never stored on a server. Transfers happen directly between the sender's and recipient's machines. The only infrastructure required is a static landing page that routes the recipient to their download.

Core Principles

- **Serverless first.** No file ever touches LinkaBoo infrastructure. The relay page is static HTML.
- **As few steps as possible.** Drag-to-send is the primary interaction. Everything else is secondary.

- **Extensible architecture.** Every core action is exposed in a way that Raycast, Alfred, CLI, or future platforms can consume.
 - **Honest about limitations.** If the sender goes offline, we say so clearly rather than showing an error page.
 - **Delightful, not cutesy.** The ghost character (Boo) adds personality at key moments — sending, receiving, success — not everywhere.
-

How It Works (Technical Foundation)

LinkaBoo uses **Magic Wormhole** (via the `wormhole-william` Go library) for all transfers.

Transfer Flow

1. Sender selects a file/folder (drag or right-click)
2. App generates a one-time wormhole code (e.g. `7-crossword-clockwork`)
3. App constructs a branded link: `https://linkaboo.app/r/7-crossword-clockwork`
4. Link is copied to clipboard. Sender shares it however they want (text, email, Slack, etc.)
5. Recipient opens the link in any browser
6. If they have LinkaBoo: deep link opens the app, transfer begins automatically
7. If they don't: landing page offers "Receive in Browser" (redirects to `wormhole.app` with code pre-filled) or "Download LinkaBoo"
8. Both parties must be online simultaneously — this is by design, not a bug

The "Host Offline" State

If the sender's machine is asleep or the app is closed when the recipient opens the link, the landing page shows:

"The sender appears to be offline. The file is only available when their computer is active. Contact them to let them know you're ready to receive."

This is honest, expected behavior. We surface it clearly rather than hiding it.

Application Architecture

Components

Component	Language	Purpose
Core Transfer Engine	Go (<code>wormhole-william</code>)	Magic Wormhole send/receive
macOS Menubar App	SwiftUI	Native UI, system integration
Finder Extension	Swift	Right-click context menu
Landing Page	Static HTML (Vercel/Netlify)	Recipient entry point, deep link routing
CLI Interface	Go	Headless transfer, Raycast/Alfred integration

What Lives on a Server

Only one thing: a static HTML page at `linkaboo.app/r/[code]` . It contains no database, no file storage, no user accounts. It reads the code from the URL and routes accordingly.

User Flows

Flow 1: Drag to Send

1. User drags a file or folder onto the desktop or toward the dock
2. LinkaBoo widget appears anchored near the app bar (active drop target)
3. User drops the file onto the widget
4. Email/name input appears — minimal, centered, one field
5. User types recipient info, presses Enter
6. Confirmation dialog appears: ghost icon, file name, recipient, "Share" button
7. User hits Share
8. Transfer initiates — Boo animation: holding folder, bobbing
9. Link is generated and copied to clipboard

10. Toast: "Link copied to clipboard"
11. Boo waves, flies off screen, reappears and dances
12. Transfer row is added to history

Flow 2: Right-Click to Send

1. User right-clicks any file or folder in Finder
2. Context menu shows "Share with LinkaBoo"
3. LinkaBoo opens (or comes to foreground)
4. Continues from step 4 of Flow 1

Flow 3: Receive (Has LinkaBoo)

1. Recipient taps/clicks the link
2. Browser triggers deep link: `linkaboo://receive/7-crossword-clockwork`
3. App opens and begins receiving automatically
4. Progress shown with Boo animation
5. File saved to Downloads (or user-configured location)
6. Toast: "File received from [name]"

Flow 4: Receive (No LinkaBoo)

1. Recipient opens `linkaboo.app/r/7-crossword-clockwork` in browser
2. Landing page shows: file name (if available), sender context, two CTAs
 - **"Receive File"** → redirects to `wormhole.app` with code pre-filled
 - **"Download LinkaBoo"** → goes to download/marketing page
3. Recipient completes transfer through `wormhole.app`
4. Landing page note: "Both computers must be online at the same time for transfer to complete"

Screen Inventory

macOS App

Screen	Description	Status
Idle / Menubar Icon	Ghost icon in menubar, no active state	Designed
Drop Target Widget	Appears on drag, anchored near dock	Designed
Email Input	Minimal centered input with folder icon	Designed
Confirmation Dialog	Ghost + file info + recipient + Share CTA	Designed
Transfer In Progress	Boo holding folder, cancel link	Designed
Transfer Success	Boo dancing, toast with link copied	Designed
History Screen	List of sent files with recipient, timestamp	Designed
Receive Notification	System notification: "[Name] shared [file]"	Designed
Receive In Progress	Boo animating, progress indicator	Needs Design
Settings	Transfer preferences, download location, account	Needs Design
Onboarding / First Launch	What is LinkaBoo, permissions setup	Needs Design
Error States	Host offline, transfer failed, cancelled	Needs Design

Landing Page (Web)

State	Description
Active — Sender Online	File ready, "Receive" CTA prominent
Offline — Sender Unreachable	Clear message, contact instructions
Expired / Invalid Code	Code no longer valid
Download LinkaBoo CTA	Secondary to the receive action

History Screen

Accessible from the menubar icon. Shows all sent transfers.

Columns: File/Folder name · File type + count · Recipient · Timestamp · Status

Status values:

- Waiting — link generated, recipient hasn’t opened it
- In Progress — transfer actively running
- Completed — transfer finished successfully
- Cancelled — either party cancelled
- Expired — code is no longer valid

Hover state on any row: “Revoke Link” option appears. Since wormhole codes are one-time, revoking just marks the record in local history — the code itself expires naturally after use or timeout.

Boo — Animation States

Boo is the ghost character. He appears at key emotional moments only, not as a persistent UI element.

Trigger	Animation
Drop target active	Boo peeks up from dock area
Transfer initiating	Boo holds folder, bobs with anticipation
Transfer in progress	Boo walking/carrying folder
Transfer complete (sender)	Boo waves, flies off, reappears and dances
Transfer complete (receiver)	Boo delivers folder, does a little bow
Error / offline	Boo looks worried, shrugs

Expression library uses the full emotion set shown in brand assets — happy, excited, winking, neutral, surprised — mapped to app states.

Settings (Required for MVP)

- **Default download location** (default: ~/Downloads)
 - **Launch at login** toggle
 - **Menubar icon style** (ghost variants)
 - **Transfer history** — clear history, set retention period
 - **Deep link handling** — confirm that linkaboo:// is registered
 - **About / Version** — links to changelog, feedback
-

Extension Points (v1 Architecture Must Support)

The following are not v1 features but the architecture must not preclude them:

Raycast Extension

- `linkaboo send [file] [recipient]` — trigger send from Raycast
- `linkaboo history` — view recent transfers in Raycast
- Uses the Go CLI layer directly

Alfred Workflow

Same as Raycast, different interface layer

CLI

```
linkaboo send ./my-file.zip
# Returns: linkaboo.app/r/7-crossword-clockwork
# Copies to clipboard automatically

linkaboo receive 7-crossword-clockwork
# Begins receiving, saves to current directory
```

iOS / iPadOS (Future)

Share sheet extension that accepts files from Files.app and any share target

What LinkaBoo Is Not (v1 Scope Boundary)

- Not a cloud storage service
 - Not an async file transfer service (both parties must be online)
 - Not a contact management system (v2)
 - Not a messaging platform
 - Not end-to-end encrypted with key management UI (Magic Wormhole handles encryption by protocol)
 - Does not require user accounts or sign-in
-

v2 Considerations (Do Not Build Now)

These require backend infrastructure and are explicitly out of scope:

- Push notifications to recipients when a file is ready
- Async transfers (sender offline, file still available)
- Contact list / address book
- Team sharing / org accounts
- Transfer analytics
- Persistent shareable links

When v2 is pursued, the architecture upgrade path is: add a lightweight relay server (Fly.io, ~\$5/mo) that receives from the wormhole and serves a temporary download. This unlocks async without changing the client UX.

Branding

- **Name:** LinkaBoo / LINKA.BOO
- **Domain:** linkaboo.app
- **Character:** Boo (ghost mascot)
- **Primary color:** Electric blue #2B2BFF

- **Secondary:** Deep navy, white, yellow accent (Boo's tongue)
 - **Typography:** Rodger Black Italic (wordmark), system font (UI)
 - **Tone:** Playful but not precious. Fast and reliable first, delightful second.
-

Open Questions

1. **Code sharing UX:** Do we auto-open the user's default mail client with a pre-written message, or just copy the link and let them paste it? Recommend: copy + optional "Open in Mail" button.
 2. **Transfer timeout:** How long does a wormhole code stay valid before expiring? Default in wormhole-william is configurable — needs a defined default (recommend 10 minutes).
 3. **File size limits:** No server-side limit, but large transfers over slow connections need a UX consideration. Show estimated time?
 4. **Simultaneous transfers:** Can a user send to multiple people at once? Each gets its own code, but does the UI support multiple in-flight transfers?
 5. **Permissions flow:** Finder Extension requires explicit user permission on macOS. Onboarding must guide the user through this or the right-click flow won't work.
-

Development Priority Order (MVP)

1. Core Go transfer engine (complete)
2. Landing page — static HTML on Vercel with online/offline states
3. macOS menubar app — drag-to-send flow end to end
4. Finder Extension — right-click integration
5. History screen
6. Receive flow (has app)
7. Settings screen
8. Onboarding / first launch
9. Error states
10. Boo animations (can ship with static image first, animate in polish pass)

